It passes with the following:

display/02/app/helpers/projects_helper.rb

```
module ProjectsHelper
  def name_with_status(project)
    dom_class = project.on_schedule? ? "on_schedule" : "behind_schedule"
    content_tag(:span, project.name, class: dom_class)
  end
end
```

One gotcha that you need to worry about when testing helpers is using Rails-internal view methods like url_for. Although all core Rails helpers are automatically loaded into the ActionView test environment, one or two have significant dependencies on the real controller object and therefore fail with opaque error messages during helper testing. The most notable of these is url_for. One workaround is to override url_for by defining it in your own test case. (The method signature is def url_for(options = {}).) The return value is up to you; a simple stub response is often good enough.

Sometimes helper methods take a block, which is expected to be ERB text (or the text of whatever template tool you're using to replace ERB). One common use of this kind of helper is access control, in which the logic in the helper determines whether the code in the block is invoked. Blocks also are very helpful as wrapper code for HTML that might surround many different kinds of text—a rounded-rectangle effect, for example.

Here's a simple example of a helper that takes a block:

```
def if_logged_in
  yield if logged_in?
end
```

It would be invoked like so:

```
<% if_logged_in do %>
 <%= link_to "logout", logout_path %>
<% end %>
```

To test the if_logged_in helper, you can take advantage of the fact that the yield statement is the last statement of the helper and, therefore, is the return value, and of the fact that Ruby will let you pass any arbitrary string into the block, giving you tests that look like this:

```
it "does not display if not logged_in" do
  expect(logged_in?).to be_falsy
  expect(if_logged_in { "logged in" }).to be_nil
end
```